Batch 26 — Enforcement Engine Deep Integration Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:27

Batch 26 — Enforcement Engine Deep Integration Pack

Meat Lovers CIMS powered by YohPal

26A.

database/migrations/059_staff_incidents.sql

```
CREATE TABLE staff_incidents (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    staff_id BIGINT NOT NULL,
    incident_date DATE NOT NULL,

    incident_type ENUM(
        'CASH_VARIANCE',
        'STOCK_VARIANCE',
        'ATTENDANCE_VIOLATION',
        'PRICING_VIOLATION',
        'UNAUTHORIZED_DISCOUNT',
        'ORDER_CANCELLATION',
        'CUSTOMER_COMPLAINT',
        'ASSET_DAMAGE',
        'GENERAL_MISCONDUCT'
    ) NOT NULL,

    severity ENUM(
        'LOW',
        'MEDIUM',
        'HIGH',
        'CRITICAL'
    ) DEFAULT 'LOW',

    description TEXT NOT NULL,

    incident_status ENUM(
        'OPEN',
        'UNDER_REVIEW',
        'RESOLVED',
        'CLOSED'
    ) DEFAULT 'OPEN',

    reported_by BIGINT,

    FOREIGN KEY (staff_id) REFERENCES users(id),
    FOREIGN KEY (reported_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

26B.

database/migrations/060_enforcement_risk_scores.sql

```
CREATE TABLE enforcement_risk_scores (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    staff_id BIGINT NOT NULL,

    risk_date DATE NOT NULL,

    audit_score DECIMAL(12,2) DEFAULT 0,
    approval_score DECIMAL(12,2) DEFAULT 0,
    stock_variance_score DECIMAL(12,2) DEFAULT 0,
    cash_variance_score DECIMAL(12,2) DEFAULT 0,
    attendance_score DECIMAL(12,2) DEFAULT 0,
    pricing_violation_score DECIMAL(12,2) DEFAULT 0,
```

```
    incident_score DECIMAL(12,2) DEFAULT 0,

    total_risk_score DECIMAL(12,2) DEFAULT 0,

    risk_level ENUM(
        'LOW',
        'MEDIUM',
        'HIGH',
        'CRITICAL'
    ) DEFAULT 'LOW',

    notes TEXT,

    FOREIGN KEY (staff_id) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

26C.

database/migrations/061_enforcement_actions.sql

```
CREATE TABLE enforcement_actions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    staff_id BIGINT NOT NULL,

    action_type ENUM(
        'VERBAL_WARNING',
        'WRITTEN_WARNING',
        'DEDUCTION_REVIEW',
        'SUSPENSION_REVIEW',
        'MANAGER_REVIEW',
        'INVESTIGATION',
        'TRAINING_REQUIRED'
    ) NOT NULL,

    action_status ENUM(
        'PENDING',
        'IN_PROGRESS',
        'COMPLETED',
        'CANCELLED'
    ) DEFAULT 'PENDING',

    reason TEXT NOT NULL,

    assigned_to BIGINT,
    due_date DATE,

    FOREIGN KEY (staff_id) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

26D. Update

backend/routes/api.php

```
$router->get('/api/enforcement/staff-incidents', [EnforcementController::class, 'staffIncidents']);
$router->post('/api/enforcement/staff-incidents', [EnforcementController::class, 'createStaffIncident']);

$router->get('/api/enforcement/risk-scores', [EnforcementController::class, 'riskScores']);
$router->post('/api/enforcement/generate-risk-scores', [EnforcementController::class, 'generateRiskScores']);

$router->get('/api/enforcement/actions', [EnforcementController::class, 'actions']);
$router->post('/api/enforcement/actions', [EnforcementController::class, 'createAction']);

$router->get('/api/enforcement/dashboard', [EnforcementController::class, 'dashboard']);
$router->get('/api/enforcement/audit-feed', [EnforcementController::class, 'auditFeed']);
```

Add import if missing:

```
use Controllers\EnforcementController;
```

26E.

backend/app/Controllers/EnforcementController.php

```php
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Auth;
use Support\Database;
use Support\Request;
use Support\Response;
use Support\AuditLogger;

class EnforcementController
{
    public function staffIncidents(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

        $stmt = Database::connection()->query(
            "SELECT
                si.*,
                staff.full_name AS staff_name,
                reporter.full_name AS reported_by_name
             FROM staff_incidents si
             JOIN users staff ON staff.id = si.staff_id
             LEFT JOIN users reporter ON reporter.id = si.reported_by
             ORDER BY si.created_at DESC"
        );

        Response::success(['staff_incidents' => $stmt->fetchAll()]);
    }

    public function createStaffIncident(): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
        $data = Request::body();

        $stmt = Database::connection()->prepare(
            'INSERT INTO staff_incidents
            (staff_id, incident_date, incident_type, severity, description, reported_by)
            VALUES (:staff_id, :incident_date, :incident_type, :severity, :description, :reported_by)'
        );

        $stmt->execute([
            'staff_id' => $data['staff_id'],
            'incident_date' => $data['incident_date'],
            'incident_type' => $data['incident_type'],
            'severity' => $data['severity'] ?? 'LOW',
            'description' => $data['description'],
            'reported_by' => $user['id'],
        ]);

        AuditLogger::log(
            (int) $user['id'],
            'ENFORCEMENT',
            'CREATE_STAFF_INCIDENT',
            (int) $data['staff_id'],
            null,
            $data
        );

        Response::success([], 'Staff incident created');
    }

    public function riskScores(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

        $stmt = Database::connection()->query(
            "SELECT
                ers.*,
                u.full_name AS staff_name,
                u.role
             FROM enforcement_risk_scores ers
             JOIN users u ON u.id = ers.staff_id
             ORDER BY ers.risk_date DESC, ers.total_risk_score DESC"
        );

        Response::success(['risk_scores' => $stmt->fetchAll()]);
    }

    public function generateRiskScores(): void
    {
        $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
        $db = Database::connection();

        $staffStmt = $db->query(
```

```
            "SELECT id, role FROM users
             WHERE role IN ('WAITER', 'CASHIER', 'CHEF', 'STOREKEEPER', 'BARMAN', 'DISPATCHER', 'ACCOUNTANT',
'HR')
             AND is_active = 1"
        );

        $staffMembers = $staffStmt->fetchAll();
        $created = 0;

        foreach ($staffMembers as $staff) {
            $staffId = (int) $staff['id'];

            $auditScore = $this->countAuditActions($staffId) * 1;
            $approvalScore = $this->countApprovalRequests($staffId) * 3;
            $stockVarianceScore = $this->countStockVarianceRelated($staffId) * 5;
            $cashVarianceScore = $this->countCashVarianceRelated($staffId) * 5;
            $attendanceScore = $this->countAttendanceViolations($staffId) * 4;
            $pricingViolationScore = $this->countPricingViolations($staffId) * 5;
            $incidentScore = $this->incidentSeverityScore($staffId);

            $total = $auditScore
                + $approvalScore
                + $stockVarianceScore
                + $cashVarianceScore
                + $attendanceScore
                + $pricingViolationScore
                + $incidentScore;

            $riskLevel = $this->riskLevel($total);

            $insert = $db->prepare(
                'INSERT INTO enforcement_risk_scores
                (staff_id, risk_date, audit_score, approval_score, stock_variance_score,
                 cash_variance_score, attendance_score, pricing_violation_score,
                 incident_score, total_risk_score, risk_level, notes)
                VALUES
                (:staff_id, CURDATE(), :audit_score, :approval_score, :stock_score,
                 :cash_score, :attendance_score, :pricing_score,
                 :incident_score, :total_score, :risk_level, :notes)'
            );

            $insert->execute([
                'staff_id' => $staffId,
                'audit_score' => $auditScore,
                'approval_score' => $approvalScore,
                'stock_score' => $stockVarianceScore,
                'cash_score' => $cashVarianceScore,
                'attendance_score' => $attendanceScore,
                'pricing_score' => $pricingViolationScore,
                'incident_score' => $incidentScore,
                'total_score' => $total,
                'risk_level' => $riskLevel,
                'notes' => 'Generated by Enforcement Engine',
            ]);

            $created++;
        }

        AuditLogger::log(
            (int) $user['id'],
            'ENFORCEMENT',
            'GENERATE_RISK_SCORES',
            null,
            null,
            ['created' => $created]
        );

        Response::success(['created' => $created], 'Risk scores generated');
    }

    public function actions(): void
    {
        Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

        $stmt = Database::connection()->query(
            "SELECT
                ea.*,
                staff.full_name AS staff_name,
                assignee.full_name AS assigned_to_name
             FROM enforcement_actions ea
             JOIN users staff ON staff.id = ea.staff_id
             LEFT JOIN users assignee ON assignee.id = ea.assigned_to
             ORDER BY ea.created_at DESC"
        );

        Response::success(['enforcement_actions' => $stmt->fetchAll()]);
    }
```

```
public function createAction(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO enforcement_actions
        (staff_id, action_type, action_status, reason, assigned_to, due_date)
        VALUES (:staff_id, :action_type, "PENDING", :reason, :assigned_to, :due_date)'
    );

    $stmt->execute([
        'staff_id' => $data['staff_id'],
        'action_type' => $data['action_type'],
        'reason' => $data['reason'],
        'assigned_to' => $data['assigned_to'] ?? $user['id'],
        'due_date' => $data['due_date'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'ENFORCEMENT',
        'CREATE_ENFORCEMENT_ACTION',
        (int) $data['staff_id'],
        null,
        $data
    );

    Response::success([], 'Enforcement action created');
}

public function dashboard(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $db = Database::connection();

    $summary = $db->query(
        "SELECT
            COUNT(*) AS scored_staff,
            SUM(CASE WHEN risk_level = 'LOW' THEN 1 ELSE 0 END) AS low_risk,
            SUM(CASE WHEN risk_level = 'MEDIUM' THEN 1 ELSE 0 END) AS medium_risk,
            SUM(CASE WHEN risk_level = 'HIGH' THEN 1 ELSE 0 END) AS high_risk,
            SUM(CASE WHEN risk_level = 'CRITICAL' THEN 1 ELSE 0 END) AS critical_risk
         FROM enforcement_risk_scores
         WHERE risk_date = CURDATE()"
    )->fetch();

    $openIncidents = $db->query(
        "SELECT COUNT(*) AS total
         FROM staff_incidents
         WHERE incident_status IN ('OPEN', 'UNDER_REVIEW')"
    )->fetch();

    $pendingActions = $db->query(
        "SELECT COUNT(*) AS total
         FROM enforcement_actions
         WHERE action_status IN ('PENDING', 'IN_PROGRESS')"
    )->fetch();

    $topRisk = $db->query(
        "SELECT
            ers.*,
            u.full_name,
            u.role
         FROM enforcement_risk_scores ers
         JOIN users u ON u.id = ers.staff_id
         WHERE ers.risk_date = CURDATE()
         ORDER BY ers.total_risk_score DESC
         LIMIT 10"
    )->fetchAll();

    Response::success([
        'summary' => $summary,
        'open_incidents' => (int) $openIncidents['total'],
        'pending_actions' => (int) $pendingActions['total'],
        'top_risk_staff' => $topRisk,
    ]);
}

public function auditFeed(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'HR']);

    $stmt = Database::connection()->query(
        "SELECT
```

```
            al.*,
            u.full_name AS user_name
        FROM audit_logs al
        LEFT JOIN users u ON u.id = al.user_id
        WHERE al.module_name IN (
            'PAYMENTS',
            'INVENTORY',
            'STOREKEEPING',
            'APPROVALS',
            'HRM',
            'PRICING',
            'ASSETS',
            'ENFORCEMENT'
        )
        ORDER BY al.created_at DESC
        LIMIT 200"
    );

    Response::success(['audit_feed' => $stmt->fetchAll()]);
}

private function countAuditActions(int $staffId): int
{
    $stmt = Database::connection()->prepare(
        'SELECT COUNT(*) AS total
         FROM audit_logs
         WHERE user_id = :staff_id
         AND DATE(created_at) = CURDATE()'
    );

    $stmt->execute(['staff_id' => $staffId]);
    $row = $stmt->fetch();

    return (int) $row['total'];
}

private function countApprovalRequests(int $staffId): int
{
    $stmt = Database::connection()->prepare(
        'SELECT COUNT(*) AS total
         FROM approval_requests
         WHERE requested_by = :staff_id
         AND approval_status = "PENDING"'
    );

    $stmt->execute(['staff_id' => $staffId]);
    $row = $stmt->fetch();

    return (int) $row['total'];
}

private function countStockVarianceRelated(int $staffId): int
{
    $stmt = Database::connection()->prepare(
        "SELECT COUNT(*) AS total
         FROM staff_incidents
         WHERE staff_id = :staff_id
         AND incident_type = 'STOCK_VARIANCE'
         AND incident_status IN ('OPEN', 'UNDER_REVIEW')"
    );

    $stmt->execute(['staff_id' => $staffId]);
    $row = $stmt->fetch();

    return (int) $row['total'];
}

private function countCashVarianceRelated(int $staffId): int
{
    $stmt = Database::connection()->prepare(
        "SELECT COUNT(*) AS total
         FROM staff_incidents
         WHERE staff_id = :staff_id
         AND incident_type = 'CASH_VARIANCE'
         AND incident_status IN ('OPEN', 'UNDER_REVIEW')"
    );

    $stmt->execute(['staff_id' => $staffId]);
    $row = $stmt->fetch();

    return (int) $row['total'];
}

private function countAttendanceViolations(int $staffId): int
{
    $stmt = Database::connection()->prepare(
        "SELECT COUNT(*) AS total
```

```
             FROM staff_attendance
             WHERE staff_id = :staff_id
             AND attendance_date = CURDATE()
             AND attendance_status IN ('LATE', 'ABSENT', 'LEFT_EARLY')"
        );

        $stmt->execute(['staff_id' => $staffId]);
        $row = $stmt->fetch();

        return (int) $row['total'];
    }

    private function countPricingViolations(int $staffId): int
    {
        $stmt = Database::connection()->prepare(
            'SELECT COUNT(*) AS total
             FROM price_change_audit
             WHERE changed_by = :staff_id
             AND DATE(created_at) = CURDATE()'
        );

        $stmt->execute(['staff_id' => $staffId]);
        $row = $stmt->fetch();

        return (int) $row['total'];
    }

    private function incidentSeverityScore(int $staffId): int
    {
        $stmt = Database::connection()->prepare(
            "SELECT severity, COUNT(*) AS total
             FROM staff_incidents
             WHERE staff_id = :staff_id
             AND incident_status IN ('OPEN', 'UNDER_REVIEW')
             GROUP BY severity"
        );

        $stmt->execute(['staff_id' => $staffId]);
        $rows = $stmt->fetchAll();

        $score = 0;

        foreach ($rows as $row) {
            $weight = match ($row['severity']) {
                'LOW' => 2,
                'MEDIUM' => 5,
                'HIGH' => 10,
                'CRITICAL' => 20,
                default => 1,
            };

            $score += $weight * (int) $row['total'];
        }

        return $score;
    }

    private function riskLevel(float $score): string
    {
        if ($score >= 50) {
            return 'CRITICAL';
        }

        if ($score >= 30) {
            return 'HIGH';
        }

        if ($score >= 15) {
            return 'MEDIUM';
        }

        return 'LOW';
    }
}
```

26F. Update

apps/admin-web/src/features/operations/api/operationsApi.js

```
enforcementDashboard: async () => {
  const { data } = await apiClient.get('/enforcement/dashboard')
  return data.data
},
```

```
staffIncidents: async () => {
  const { data } = await apiClient.get('/enforcement/staff-incidents')
  return data.data.staff_incidents
},

createStaffIncident: async (payload) => {
  const { data } = await apiClient.post('/enforcement/staff-incidents', payload)
  return data
},

riskScores: async () => {
  const { data } = await apiClient.get('/enforcement/risk-scores')
  return data.data.risk_scores
},

generateRiskScores: async () => {
  const { data } = await apiClient.post('/enforcement/generate-risk-scores', {})
  return data
},

enforcementActions: async () => {
  const { data } = await apiClient.get('/enforcement/actions')
  return data.data.enforcement_actions
},

createEnforcementAction: async (payload) => {
  const { data } = await apiClient.post('/enforcement/actions', payload)
  return data
},

enforcementAuditFeed: async () => {
  const { data } = await apiClient.get('/enforcement/audit-feed')
  return data.data.audit_feed
},
```

26G.

apps/admin-web/src/features/enforcement/pages/EnforcementDashboardPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function EnforcementDashboardPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['enforcement-dashboard'],
    queryFn: operationsApi.enforcementDashboard,
    refetchInterval: 15000,
  })

  return (
    <Page title="Enforcement Risk Dashboard" subtitle="Unified control across audit, approvals, stock, cash,
HRM, pricing, and incidents">
      {isLoading ? <LoadingBlock label="Loading enforcement dashboard..." /> : null}
      {error ? <StatusMessage error={error.message} /> : null}

      {data ? (
        <>
          <div style={{ display: 'grid', gridTemplateColumns: 'repeat(7, minmax(0, 1fr))', gap: 16 }}>
            <Card title="Scored Staff"><strong>{data.summary.scored_staff}</strong></Card>
            <Card title="Low Risk"><strong>{data.summary.low_risk}</strong></Card>
            <Card title="Medium Risk"><strong>{data.summary.medium_risk}</strong></Card>
            <Card title="High Risk"><strong>{data.summary.high_risk}</strong></Card>
            <Card title="Critical Risk"><strong>{data.summary.critical_risk}</strong></Card>
            <Card title="Open Incidents"><strong>{data.open_incidents}</strong></Card>
            <Card title="Pending Actions"><strong>{data.pending_actions}</strong></Card>
          </div>

          <Card title="Top Risk Staff">
            <DataTable
              columns={[
                { key: 'full_name', label: 'Staff' },
                { key: 'role', label: 'Role' },
                { key: 'total_risk_score', label: 'Risk Score' },
                { key: 'risk_level', label: 'Risk Level' },
                { key: 'audit_score', label: 'Audit' },
                { key: 'approval_score', label: 'Approvals' },
                { key: 'attendance_score', label: 'Attendance' },
                { key: 'incident_score', label: 'Incidents' },
```

```
              ]}
              rows={data.top_risk_staff || []}
            />
          </Card>
        </>
      ) : null}
    </Page>
  )
}
```

26H.

apps/admin-web/src/features/enforcement/pages/StaffIncidentsPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function StaffIncidentsPage() {
  return (
    <OperationalFormPage
      title="Staff Incidents"
      subtitle="Record misconduct, variance, attendance, pricing, and customer complaint incidents"
      queryKey="staff-incidents"
      queryFn={operationsApi.staffIncidents}
      mutationFn={operationsApi.createStaffIncident}
      initialForm={{
        staff_id: '',
        incident_date: '',
        incident_type: 'GENERAL_MISCONDUCT',
        severity: 'LOW',
        description: '',
      }}
      buildPayload={(form) => ({
        staff_id: Number(form.staff_id),
        incident_date: form.incident_date,
        incident_type: form.incident_type,
        severity: form.severity,
        description: form.description,
      })}
      fields={[
        { name: 'staff_id', label: 'Staff ID', type: 'number' },
        { name: 'incident_date', label: 'Incident Date', type: 'date' },
        {
          name: 'incident_type',
          label: 'Incident Type',
          type: 'select',
          options: [
            { value: 'CASH_VARIANCE', label: 'Cash Variance' },
            { value: 'STOCK_VARIANCE', label: 'Stock Variance' },
            { value: 'ATTENDANCE_VIOLATION', label: 'Attendance Violation' },
            { value: 'PRICING_VIOLATION', label: 'Pricing Violation' },
            { value: 'UNAUTHORIZED_DISCOUNT', label: 'Unauthorized Discount' },
            { value: 'ORDER_CANCELLATION', label: 'Order Cancellation' },
            { value: 'CUSTOMER_COMPLAINT', label: 'Customer Complaint' },
            { value: 'ASSET_DAMAGE', label: 'Asset Damage' },
            { value: 'GENERAL_MISCONDUCT', label: 'General Misconduct' },
          ],
        },
        {
          name: 'severity',
          label: 'Severity',
          type: 'select',
          options: [
            { value: 'LOW', label: 'Low' },
            { value: 'MEDIUM', label: 'Medium' },
            { value: 'HIGH', label: 'High' },
            { value: 'CRITICAL', label: 'Critical' },
          ],
        },
        { name: 'description', label: 'Description', type: 'textarea' },
      ]}
      columns={[
        { key: 'staff_name', label: 'Staff' },
        { key: 'incident_date', label: 'Date' },
        { key: 'incident_type', label: 'Type' },
        { key: 'severity', label: 'Severity' },
        { key: 'incident_status', label: 'Status' },
        { key: 'reported_by_name', label: 'Reported By' },
      ]}
      submitLabel="Record Incident"
    />
  )
}
```

26I.

apps/admin-web/src/features/enforcement/pages/RiskScoresPage.jsx

```
import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function RiskScoresPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['risk-scores'],
    queryFn: operationsApi.riskScores,
  })

  const mutation = useMutation({
    mutationFn: operationsApi.generateRiskScores,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['risk-scores'] }),
  })

  return (
    <Page title="Risk Scores" subtitle="Generated risk score per staff member">
      {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}

      <Card title="Generate Scores">
        <Button onClick={() => mutation.mutate()}>Generate Today’s Risk Scores</Button>
      </Card>

      <Card title="Risk Score History">
        {isLoading ? <LoadingBlock label="Loading risk scores..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'staff_name', label: 'Staff' },
              { key: 'role', label: 'Role' },
              { key: 'risk_date', label: 'Date' },
              { key: 'total_risk_score', label: 'Total Score' },
              { key: 'risk_level', label: 'Level' },
              { key: 'cash_variance_score', label: 'Cash' },
              { key: 'stock_variance_score', label: 'Stock' },
              { key: 'attendance_score', label: 'Attendance' },
              { key: 'pricing_violation_score', label: 'Pricing' },
              { key: 'incident_score', label: 'Incident' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

26J.

apps/admin-web/src/features/enforcement/pages/EnforcementActionsPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function EnforcementActionsPage() {
  return (
    <OperationalFormPage
      title="Enforcement Actions"
      subtitle="Create corrective actions, warnings, investigations, and training requirements"
      queryKey="enforcement-actions"
      queryFn={operationsApi.enforcementActions}
      mutationFn={operationsApi.createEnforcementAction}
      initialForm={{
        staff_id: '',
        action_type: 'MANAGER_REVIEW',
        reason: '',
        assigned_to: '',
```

```
        due_date: '',
      }}
      buildPayload={(form) => ({
        staff_id: Number(form.staff_id),
        action_type: form.action_type,
        reason: form.reason,
        assigned_to: form.assigned_to ? Number(form.assigned_to) : null,
        due_date: form.due_date,
      })}
      fields={[
        { name: 'staff_id', label: 'Staff ID', type: 'number' },
        {
          name: 'action_type',
          label: 'Action Type',
          type: 'select',
          options: [
            { value: 'VERBAL_WARNING', label: 'Verbal Warning' },
            { value: 'WRITTEN_WARNING', label: 'Written Warning' },
            { value: 'DEDUCTION_REVIEW', label: 'Deduction Review' },
            { value: 'SUSPENSION_REVIEW', label: 'Suspension Review' },
            { value: 'MANAGER_REVIEW', label: 'Manager Review' },
            { value: 'INVESTIGATION', label: 'Investigation' },
            { value: 'TRAINING_REQUIRED', label: 'Training Required' },
          ],
        },
        { name: 'reason', label: 'Reason', type: 'textarea' },
        { name: 'assigned_to', label: 'Assigned Staff ID optional', type: 'number' },
        { name: 'due_date', label: 'Due Date', type: 'date' },
      ]}
      columns={[
        { key: 'staff_name', label: 'Staff' },
        { key: 'action_type', label: 'Action' },
        { key: 'action_status', label: 'Status' },
        { key: 'reason', label: 'Reason' },
        { key: 'assigned_to_name', label: 'Assigned To' },
        { key: 'due_date', label: 'Due Date' },
      ]}
      submitLabel="Create Action"
    />
  )
}
```

26K.

apps/admin-web/src/features/enforcement/pages/EnforcementAuditFeedPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function EnforcementAuditFeedPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['enforcement-audit-feed'],
    queryFn: operationsApi.enforcementAuditFeed,
    refetchInterval: 15000,
  })

  return (
    <Page title="Enforcement Audit Feed" subtitle="Sensitive system activity feed">
      <Card>
        {isLoading ? <LoadingBlock label="Loading enforcement audit feed..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'user_name', label: 'User' },
              { key: 'module_name', label: 'Module' },
              { key: 'action_name', label: 'Action' },
              { key: 'entity_id', label: 'Entity' },
              { key: 'ip_address', label: 'IP' },
              { key: 'created_at', label: 'Date' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
```

```
}
```

26L. Update

apps/admin-web/src/app/routes.jsx

```
import EnforcementDashboardPage from '../features/enforcement/pages/EnforcementDashboardPage'
import StaffIncidentsPage from '../features/enforcement/pages/StaffIncidentsPage'
import RiskScoresPage from '../features/enforcement/pages/RiskScoresPage'
import EnforcementActionsPage from '../features/enforcement/pages/EnforcementActionsPage'
import EnforcementAuditFeedPage from '../features/enforcement/pages/EnforcementAuditFeedPage'

<Route path="/enforcement/dashboard" element={<EnforcementDashboardPage />} />
<Route path="/enforcement/incidents" element={<StaffIncidentsPage />} />
<Route path="/enforcement/risk-scores" element={<RiskScoresPage />} />
<Route path="/enforcement/actions" element={<EnforcementActionsPage />} />
<Route path="/enforcement/audit-feed" element={<EnforcementAuditFeedPage />} />
```

26M. Update

apps/admin-web/src/components/common/SidebarNav.jsx

```
['Enforcement Dashboard', '/enforcement/dashboard'],
['Staff Incidents', '/enforcement/incidents'],
['Risk Scores', '/enforcement/risk-scores'],
['Enforcement Actions', '/enforcement/actions'],
['Enforcement Audit Feed', '/enforcement/audit-feed'],
```

26N.

docs/ENFORCEMENT_ENGINE_DEEP_INTEGRATION_PACK.md

````
# Batch 26 — Enforcement Engine Deep Integration Pack

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch connects enforcement into one risk engine.

## Inputs Connected

1. Audit logs
2. Approval requests
3. Stock variance incidents
4. Cash variance incidents
5. Attendance violations
6. Product pricing violations
7. Staff incident records

## Risk Score Components

Each staff risk score includes:

- audit score
- approval score
- stock variance score
- cash variance score
- attendance score
- pricing violation score
- incident score
- total risk score
- risk level

## Risk Levels

```text
LOW: 0–14
MEDIUM: 15–29
HIGH: 30–49
CRITICAL: 50+
````

Enforcement Actions

Supported actions:

- verbal warning
````

- written warning
- deduction review
- suspension review
- manager review
- investigation
- training required

Dashboard

The dashboard shows:

- scored staff
- low risk staff
- medium risk staff
- high risk staff
- critical risk staff
- open incidents
- pending actions
- top risk staff

Business Value

This helps Meat Lovers:

- stop theft
- detect repeated violations
- monitor sensitive actions
- connect HRM discipline to operational events
- protect stock, cash, pricing, assets, and service standards

---

Batch 26 Outcome

Enforcement Engine now connects:

✅ audit logs
✅ approvals
✅ stock variance
✅ cash variance
✅ attendance violations
✅ product pricing violations
✅ staff incident records
✅ unified risk-score dashboard

Next Smart Move

Build **Batch 27 — Local Deployment + Installer Pack**, including local server folder layout, Apache/Nginx config, `.env` templates, MySQL setup, frontend build commands, backend startup instructions, backup scripts, and LAN access guide.